

NORME INTERNE OBLIGATOIRE



Qalami Secure Build Standard

Obfuscation, protection des secrets & scan avant production
— le standard à appliquer à la lettre sur toute application Qalami —

Version 1.0

Audience : développeurs Flutter, web (Next.js / React / Vite), DevOps, lead tech
Origine : OWASP MASVS/ASVS, NIST, benchmark fintech, et l'ensemble des audits internes
Couverture gratuite : script kiddie → reverse engineer freelance → fuite de données

Les 3 règles non négociables

Aucune application ne part en production si elle viole **une seule** de ces règles. Le pipeline automatique les vérifie et **bloque** le merge et la release. Pas de dérogation silencieuse.

1 NO SECRET — aucun secret (clé API, mot de passe, token, clé privée, URL interne sensible) dans le code source, l'historique git, ou le binaire livré.

2 NO DEBUG KEY — aucune clé de debug, keystore de debug, mode debug, source map, ou log verbeux dans un artefact de production.

3 SCAN BEFORE PROD — toute application est scannée par le pipeline automatique avant prod, et le pipeline est bloquant.

Démarrage rapide (nouveau repo, ~15 min)

```
# 1. Copier les fichiers de norme à la racine du repo
cp .gitleaks.toml .pre-commit-config.yaml .env.example .
mkdir -p .github/workflows && cp secure-gate.yml .github/workflows/
mkdir -p scripts && cp check_no_debug.sh scripts/

# 2. Activer le hook local (chaque dev, une fois)
pipx install pre-commit && pre-commit install

# 3. Scanner l'existant AVANT le premier commit conforme
gitleaks detect --source . --redact -c .gitleaks.toml

# 4. (Flutter) build durci      cp build_release.sh scripts/ ; cp proguard-rules.pro android/app/
# 5. (Web) obfuscation        cp obfuscate.postbuild.js scripts/ ; npm i -D javascript-obfuscator
```

§0 — Vérité fondatrice

Trois affirmations cadrent toute la norme. Les ignorer mène à une fausse sécurité, pire que pas de sécurité.

1. **On ne peut pas chiffrer durablement du code exécuté côté client.** L'appareil de l'utilisateur (donc de l'attaquant potentiel) doit déchiffrer le code pour l'exécuter. La clé est livrée avec l'app. Tout schéma « code chiffré » côté client est, au mieux, une obfuscation de plus — jamais une garantie cryptographique.
2. **L'obfuscation ralentit, n'empêche pas.** Elle transforme « 1 heure avec jadx » en « 2 semaines avec IDA + un expert ». C'est une mesure économique : rendre l'attaque non rentable.
3. **La seule donnée vraiment protégée est celle qui n'est pas dans l'app.** Un secret absent du binaire ne peut pas être extrait. D'où la primauté de **NO SECRET**.

Conséquence. Défense réelle = *(absence de secret) × (coût de RE élevé) × (chiffrement des données en transit et au repos) × (détection runtime)*. Cette norme impose les quatre. Ne jamais promettre « l'encryption parfaite du code » : c'est faux. Promettre « aucun secret extractible + reverse non rentable + données chiffrées » est tenable.

§1 — Modèle de menace

Acteur	Capacité	Coût	On bloque ?
Script kiddie	Frida / jadx sur device rooté, tutos	0 €	✓
Fraudeur opportuniste	Ingénierie sociale + outils publics	< 50 €	✓
Reverse engineer freelance	jadx + IDA + 1 semaine	500–5 000 €	✓ (non rentable)
Concurrent / fuite de données	Achat de dumps, logs, repos fuités	1k–50k €	✓
Crime organisé	Phishing massif + mules	10k+ €	● partiel
APT étatique	0day + bypass shielding commercial	100k+ €	✗ hors portée

La norme couvre **les 4 premiers** intégralement avec des outils **gratuits**. Au-delà : budget shielding commercial requis (hors périmètre).

§2 — RÈGLE 1 : NO SECRET

2.1 Définition d'un « secret »

Tout élément dont la divulgation crée un risque : clé/token API, mot de passe, clé privée (RSA/EC/PEM), secret de signature JWT, chaîne de connexion DB, clé symétrique, identifiants cloud, webhook signé, **et toute clé symétrique embarquée** (publique par nature dès qu'elle est dans le binaire).

2.2 Règles

#	Règle
R2.1	Aucun secret en clair dans le code, la conf versionnée, ou les commentaires.
R2.2	Aucun secret dans l'historique git. Un secret commité une fois = compromis : rotation immédiate, pas seulement suppression.
R2.3	Aucun secret dans le binaire livré (APK/IPA/bundle JS). Vérifié par scan binaire.
R2.4	Les secrets vivent dans un gestionnaire de secrets / variables d'environnement runtime — jamais dans l'artefact.
R2.5	Le client n'embarque que des clés publiques (cert pinning, RSA pub) — jamais de clé privée ni de secret partagé.
R2.6	Config par <code>.env</code> non versionné + <code>.env.exemple</code> versionné sans valeurs .

2.3 Outils gratuits imposés

Étage	Outil	Rôle
Pré-commit (local)	gitleaks	Bloque le commit contenant un secret
Historique	gitleaks detect + trufflehog	Scanne tout le passé
CI (bloquant)	gitleaks + trufflehog	Re-scan sur chaque PR
Binaire APK	apkleaks + MobSF	Secrets dans le .apk final
Bundle web	gitleaks sur dist/	Secrets dans le JS minifié

2.4 Procédure en cas de fuite

1. **Rotation immédiate** (révoquer + régénérer chez le fournisseur).
2. Retrait du code → gestionnaire de secrets.
3. Purge de l'historique si besoin (`git filter-repo`) + force-push coordonné.
4. Entrée au registre d'incidents (date + impact).

Supprimer le secret du dernier commit ne suffit pas : il reste dans l'historique et a pu être indexé. **La rotation est obligatoire.**

§3 — RÈGLE 2 : NO DEBUG

3.1 Règles transverses

#	Règle
R3.1	Release jamais signée avec une clé de debug. Clé de release hors repo (CI secret / Play App Signing).
R3.2	Aucun mode debug en prod : pas de <code>debuggable=true</code> , <code>DEBUG=True</code> , <code>NODE_ENV=development</code> .
R3.3	Aucune source map publiée. Symboles Flutter (<code>split-debug-info</code>) archivés hors-app, jamais livrés.
R3.4	Logs verbeux désactivés en release ; aucun PII en log (PIN, mot de passe, OTP, NNI/CIN, token, photo KYC).
R3.5	Endpoints/flags de debug, comptes de test, backdoors : retirés du build de prod.
R3.6	<code>allowBackup="false"</code> ; pas d'export de composants non nécessaires.

3.2 Flutter / Android

```
// android/app/build.gradle
release {
    signingConfig signingConfigs.release    // JAMAIS .debug
    debuggable false
    minifyEnabled true
    shrinkResources true
    proguardFiles getDefaultProguardFile('proguard-android-optimize.txt'),'proguard-rules.pro'
}
```

```
<!-- AndroidManifest.xml -->
android:debuggable="false"    android:allowBackup="false"
android:networkSecurityConfig="@xml/network_security_config"
android:usesCleartextTraffic="false"
```

3.3 Web

- `productionBrowserSourceMaps:false` (Next) / `build.sourcemap:false` (Vite).
- Strip des `console.*` au build (terser `drop_console`).
- Tout `NEXT_PUBLIC_*` / `VITE_*` est **public** — jamais de secret dedans.

3.4 Backend

`DEBUG=False` , `ALLOWED_HOSTS` strict, pages d'erreur génériques (pas de stack trace exposée).

§4 — RÈGLE 3 : SCAN BEFORE PROD

Un **unique pipeline** tourne sur chaque PR et chaque tag de release. Il est **bloquant**. Aucun « je merge quand même » : la protection de branche l'empêche.

Étage	Outil	Bloque si
Secrets (code + historique)	gitleaks, trufflehog	≥ 1 secret
SAST	semgrep (rulesets gratuits)	finding <code>ERROR</code>
Dépendances (SCA)	osv-scanner, npm audit, trivy	vuln CRITICAL/HIGH exploitable
Anti debug-artifact	script <code>check_no_debug.sh</code>	≥ 1 artefact de debug
Binaire mobile (release)	MobSF / apkleaks	secret ou misconfig majeure
laC / Docker	trivy config, checkov	misconfig HIGH

Politique de sévérité

- **CRITICAL / HIGH** → bloquant.
- **MEDIUM** → non bloquant mais ticket obligatoire avant la prochaine release.
- **Secret détecté** → toujours bloquant.

Protection de branche (une fois par repo)

`main` / `prod` protégées : PR obligatoire, ≥ 1 review, status check `secure-gate` / `gate` requis, pas de push direct ni force-push.

§5 — Obfuscation Flutter (Dart AOT)

Le build release Flutter compile le Dart en **code machine** (`libapp.so`) — pas de bytecode lisible. Mais les **noms de symboles** et **chaînes** restent extractibles : l'obfuscation les renomme.

```
flutter build appbundle --release \  
  --obfuscate \  
  --split-debug-info=build/symbols  # symboles À ARCHIVER, JAMAIS LIVRER
```

+ R8/ProGuard (`minifyEnabled` + `shrinkResources` + `proguard-rules.pro`) sur la couche native.

Anti-tamper / RASP (gratuit)

Lib	Couvre
freeRASP (Talsec)	root, debugger, émulateur, hooks Frida, repackaging
Firebase App Check + Play Integrity	bloque APK non-officiel / device non attesté
package_info_plus + check signature	détecte la modification de l'APK au runtime

Limite honnête. Un attaquant déterminé avec IDA finit par comprendre le binaire — d'où **NO SECRET** comme filet : même compris, il ne livre rien d'exploitable.

§6 — Obfuscation Web

Le JS est lisible par nature. Défense, du moins cher au plus cher :

1. **Minification + mangle** (terser/esbuild) — par défaut en prod.
2. **Suppression des source maps** publiques — sans ça, l'obfuscation est annulée.
3. **Obfuscation avancée** (`javascript-obfuscator`) — string array, control-flow flattening, dead code, self-defending.
4. **Anti-debug** runtime (optionnel — peut casser des navigateurs, à tester).

⚠ L'obfuscation lourde **dégrade les performances**. La réserver aux bundles de logique sensible (auth, calculs métier), pas aux vendors. **Mesurer le coût**.

Ce qu'elle ne remplace pas : toute autorisation/validation de montant reste **côté serveur** ; JWT en cookie `HttpOnly; Secure; SameSite` (jamais `localStorage`) ; en-têtes CSP/HSTS obligatoires (voir Annexe H).

§7 — Chiffrement des données

L'app ne chiffre pas son code, mais elle **doit** chiffrer ses données.

Surface	Standard imposé	Proscrit
Transport	TLS 1.3 + cert pinning (SPKI SHA-256, 2 pins) + cleartext refusé	TLS < 1.2, pas de pinning
Asymétrique	RSA-OAEP-SHA256 ou ECIES	RSA-PKCS1v1.5, ECB
Symétrique	AES-256-GCM ou ChaCha20-Poly1305 (nonce unique, AAD)	AES-CBC sans MAC, nonce réutilisé
Hash mot de passe	Argon2id (ou bcrypt)	MD5, SHA-1 nu
Clés mobile	Keystore (StrongBox/TEE) / Keychain via flutter_secure_storage	clé en SharedPreferences claires
Stockage local	SQLCipher (AES-256)	SQLite clair
Clés serveur	KMS/HSM + Secret Manager	clé privée sur disque container

Interdits absolus : MD5, SHA-1, RC4, DES, 3DES, ECB, RSA-PKCS1v1.5, réutilisation de nonce GCM, crypto « maison ». Toujours une lib éprouvée.

§8 — Durcissement applicatif

Risque	Mesure imposée
Brute force PIN/login	Lock après N essais + backoff exponentiel
Vol de token	Access token ≤ 15 min + refresh + binding device
Rejeu	Nonce + timestamp signés, fenêtre ≤ 30 s
Multi-tenant	tenant_id signé dans le token, vérifié à chaque endpoint
Énumération	Rate-limit + réponses uniformes
Abus volume	Rate-limit par IP/user/endpoint + velocity checks
Forensique	Audit log append-only (trigger anti-update/delete)

§9 — Cycle de vie & gouvernance

- **Création d'un repo** : appliquer le démarrage rapide ; gate CI vert au premier commit.
- **Chaque PR** : le gate tourne, bloquant.
- **Chaque release** : remplir et signer la checklist Go/No-Go (Annexe J). Pas de signature = pas de release.
- **Rotation des clés** : RSA 12 mois, JWT signing 6 mois, mots de passe DB 3 mois, secrets tiers sur incident.
- **Dérogation** : uniquement via ticket daté, justifié, validé lead sécurité, avec date d'expiration.

§10 — Anti-patterns interdits

Présence en revue de code = rejet.

- ❌ Crypto maison.
- ❌ Secret/clé symétrique hardcodé (« c'est obfusqué » n'est pas une excuse).
- ❌ Cert pinning désactivé en debug sans flag de build (ça fuit en prod).
- ❌ `badCertificateCallback => true` ou équivalent (= pas de TLS).
- ❌ Valider montants/droits côté client uniquement.
- ❌ JWT en `localStorage`.
- ❌ Logger PIN, mot de passe, OTP, NNI/CIN, token, photo KYC.
- ❌ Source maps publiques en prod.
- ❌ Clé de debug pour signer une release.
- ❌ Désactiver/ignorer le gate CI pour « débloquer » une release.

§11 — Coût total (0 € de licence)

Brique	Outil	Coût
Secret scan	gitleaks, trufflehog	Gratuit
SAST	semgrep (rulesets communautaires)	Gratuit
SCA	osv-scanner, trivy, npm audit	Gratuit
Binaire mobile	MobSF, apkleaks	Gratuit
Obfuscation Flutter	flutter --obfuscate + R8	Gratuit
RASP	freeRASP, App Check / Play Integrity	Gratuit
Obfuscation web	terser + javascript-obfuscator	Gratuit
Stockage chiffré	flutter_secure_storage, SQLCipher	Gratuit
Total licences		0 € (hors infra KMS/Redis ~quelques €/mois)

Racine de chaque repo. Étend les ~150 règles intégrées + règles internes + allowlist anti-faux-positifs.

```
title = "Qalami Secret Detection"

[extend]
useDefault = true    # hérite des règles intégrées (AWS, GCP, Stripe, JWT, clés privées...)

[[rules]]
id = "qalami-private-key-pem"
description = "Clé privée PEM embarquée (interdite côté client)"
regex = '''-----BEGIN(?:RSA|EC|OPENSSH|DSA|PGP)?PRIVATE KEY-----'''
keywords = ["PRIVATE KEY"]

[[rules]]
id = "qalami-django-secret-key"
regex = '''(?:i)SECRET_KEY\s*[:=]\s*['"]([0-9a-zA-Z!@#%&*()_\-+=]{20,})['"]'''
keywords = ["SECRET_KEY"]

[[rules]]
id = "qalami-generic-api-key"
regex = '''(?:i)(api[_]?key|api[_]?secret|access[_]?token|auth[_]?token|client[_]?secret)\s*[:=]\s*['"]([0-9a-zA-Z_\-]{16,})['"]'''
keywords = ["api_key", "apikey", "access_token", "client_secret", "auth_token"]

[[rules]]
id = "qalami-db-connection-string"
regex = '''(?:i)(postgres|postgresql|mysql|mongodb(\+srv)?|redis)://[^\s]+:[^\s]+@'''
keywords = ["postgres://", "mysql://", "mongodb://", "redis://"]

[[rules]]
id = "qalami-hardcoded-password"
regex = '''(?:i)(password|passwd|pwd|mot_de_passe)\s*[:=]\s*['"]([^\s]{8,})['"]'''
keywords = ["password", "passwd", "mot_de_passe"]

[allowlist]
paths = [
    '''(.*)\.example$''',
    '''\.env\.example$''',
    '''(^/)?tests?/''',
    '''(^/)?fixtures?/''',
    '''(^/)?mock(s)?/''',
    '''package-lock\.json$'''
]
regexes = [
    '''(?:i)(your|example|placeholder|changeme|xxxx+|dummy|fake|test)[_]?(?:api|key|token|secret|password)'''
]
```

Annexe B — .pre-commit-config.yaml

Racine de chaque repo. `pipx install pre-commit && pre-commit install`. Vérifier/bumper les `rev`: à l'adoption.

```
repos:
- repo: https://github.com/gitleaks/gitleaks
  rev: v8.21.2
  hooks: [ { id: gitleaks, args: ["--config",".gitleaks.toml"] } ]

- repo: https://github.com/pre-commit/pre-commit-hooks
  rev: v5.0.0
  hooks:
    - { id: detect-private-key }
    - { id: check-added-large-files, args: ["--maxkb=2048"] }
    - { id: check-merge-conflict }
    - { id: end-of-file-fixer }
    - { id: trailing-whitespace }

- repo: local
  hooks:
    - id: no-debug-artifact
      name: "Pas d'artefact de debug"
      entry: bash scripts/check_no_debug.sh
      language: system
      pass_filenames: false
      always_run: true

- repo: https://github.com/returntocorp/semgrep
  rev: v1.95.0
  hooks:
    - { id: semgrep, args: ["--config","p/secrets","--config","p/owasp-top-ten","--error","--quiet"] }
```


Annexe C — Gate CI `.github/workflows/secure-gate.yml`

Bloquant. Exiger le check `secure-gate / gate` en protection de branche.

```
name: secure-gate
on:
  pull_request: { branches: [main, master, prod] }
  push: { branches: [main, master, prod], tags: ["v*"] }
permissions: { contents: read }

jobs:
  secrets:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with: { fetch-depth: 0 } # historique complet
      - uses: gitleaks/gitleaks-action@v2
        env: { GITLEAKS_CONFIG: .gitleaks.toml }
      - uses: trufflesecurity/trufflehog@main
        with: { extra_args: --results=verified,unknown --fail }

  sast:
    runs-on: ubuntu-latest
    container: semgrep/semgrep
    steps:
      - uses: actions/checkout@v4
      - run: semgrep ci --config p/owasp-top-ten --config p/secrets
        --config p/javascript --config p/typescript
        --config p/dart --config p/python --error

  dependencies:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: google/osv-scanner-action@v1
        with: { scan-args: "--recursive --skip-git ." }
      - uses: aquasecurity/trivy-action@master
        with: { scan-type: fs, scanners: "vuln,misconfig,secret",
          severity: "CRITICAL,HIGH", exit-code: "1", ignore-unfixed: true }

  no-debug-artifact:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: bash scripts/check_no_debug.sh

  gate:
    runs-on: ubuntu-latest # ← check exigé en protection de branche
    needs: [secrets, sast, dependencies, no-debug-artifact]
    if: always()
    steps:
      - run: |
          if [ "${{ needs.secrets.result }}" != "success" ] || \
            [ "${{ needs.sast.result }}" != "success" ] || \
            [ "${{ needs.dependencies.result }}" != "success" ] || \
            [ "${{ needs.no-debug-artifact.result }}" != "success" ]; then
            echo "❌ Secure gate FAILED"; exit 1; fi
          echo "✅ Secure gate PASSED"
```

Annexe D — scripts/check_no_debug.sh

Détecteur d'artefacts de debug, partagé par le hook et la CI. Testé : cas propre = exit 0, cas sale = exit 1.

```
#!/usr/bin/env bash
set -uo pipefail
FAIL=0
EXCLUDES='-not -path "*/node_modules/*" -not -path "*/.git/*" -not -path "*/build/*" \
          -not -path "*/dist/*" -not -path "*/.next/*" -not -name "check_no_debug.sh"'

scan() { # $1 label $2 regex $3 globs find
  local hits
  hits=$(eval find . $EXCLUDES -type f \( $3 \) -print0 2>/dev/null \
    | xargs -0 grep -HnIE "$2" 2>/dev/null)
  [ -n "$hits" ] && { echo "❌ [$1]"; echo "$hits" | sed 's/^/ /'; FAIL=1; }
}

scan "android-debuggable" 'android:debuggable\s*=\s*"true"' '-name "*.xml" -o -name "*.gradle"'
scan "android-allowBackup" 'android:allowBackup\s*=\s*"true"' '-name "*.xml"'
scan "android-debug-sign" 'signingConfig\s+signingConfigs\.debug' '-name "*.gradle"'
scan "django-debug-true" '^s*DEBUG\s*=\s*True' '-name "*.py"'
scan "next-sourcemaps" 'productionBrowserSourceMaps\s*:\s*true' '-name "next.config.*"'
scan "vite-sourcemaps" 'sourcemap\s*:\s*true' '-name "vite.config.*"'
scan "console-log" 'console\.(log|debug|info)\(' '-name "*.ts" -o -name "*.tsx" -o -name "*.js" -o -name "*.jsx"'
scan "debug-backdoor" '(BYPASS_AUTH|SKIP_AUTH|DISABLE_SECURITY|DEV_TOKEN)' '-name "*.*)"

[ "$FAIL" -ne 0 ] && { echo "Artefact(s) de debug détecté(s) – voir $3."; exit 1; }
echo "✅ Aucun artefact de debug détecté."; exit 0
```

Annexe E — Flutter `build_release.sh`

```
#!/usr/bin/env bash
set -euo pipefail
TARGET=${1:-appbundle}
SYMBOLS_DIR="build/symbols" # symboles de debug – À ARCHIVER, JAMAIS LIVRER

grep -rIE 'signingConfig\s+signingConfigs\.debug' android/app/build.gradle 2>/dev/null \
  && { echo "❌ Release signée avec la clé de debug – interdit (§3.1)"; exit 1; }

flutter clean && flutter pub get
FLAGS="--release --obfuscate --split-debug-info=${SYMBOLS_DIR}"
case "$TARGET" in
  apk) flutter build apk $FLAGS; ARTIFACT="build/app/outputs/flutter-apk/app-release.apk";;
  appbundle) flutter build appbundle $FLAGS; ARTIFACT="build/app/outputs/bundle/release/app-release.aab";;
esac

command -v apkleaks >/dev/null && [ "$TARGET" = "apk" ] && \
  apkleaks -f "$ARTIFACT" -o build/apkleaks.txt # RELIRE : tout secret = release bloquée
echo "✅ Build durci terminé."
```

```
# Garder l'engine Flutter
-keep class io.flutter.** { *; }
-keep class io.flutter.plugins.** { *; }
-dontwarn io.flutter.embedding.**

# Obfuscation agressive + suppression du code mort
-optimizationpasses 5
-allowaccessmodification
-repackageclasses ''
-mergeinterfacesaggressively

# Supprimer les logs en release (§3.4)
-assumenosideeffects class android.util.Log {
    public static *** v(...); public static *** d(...); public static *** i(...);
}

-keepattributes Signature, *Annotation*, EnclosingMethod, InnerClasses
-keepclassmembers enum * { *; }
# Modèles (de)sérialisés : décommenter et adapter le package, sinon le JSON casse
# -keep class mr.exemple.app.models.** { *; }
```

Annexe G — Web obfuscate.postbuild.js

```
npm i -D javascript-obfuscator ; "build": "vite build && node scripts/obfuscate.postbuild.js dist" .
```

```
const fs=require('fs'), path=require('path');
const Obf=require('javascript-obfuscator');
const dir=process.argv[2]; if(!dir){console.error('usage:

');process.exit(1);}
const SKIP=[/vendor/i,/runtime/i,/polyfill/i,/\.min\.js$/i];
const isJs=f=>f.endsWith('.js')&&!f.endsWith('.map');
const OPT={compact:true, controlFlowFlattening:true, controlFlowFlatteningThreshold:0.5,
  deadCodeInjection:true, stringArray:true, stringArrayEncoding:['base64'],
  stringArrayThreshold:0.75, splitStrings:true, identifierNamesGenerator:'hexadecimal',
  selfDefending:true, disableConsoleOutput:true, sourceMap:false};
function walk(d){for(const e of fs.readdirSync(d,{withFileTypes:true}))){
  const f=path.join(d,e.name);
  if(e.isDirectory()){walk(f);continue;}
  if(e.name.endsWith('.map')){fs.unlinkSync(f);continue;} // jamais de source map (§3.3)
  if(!isJs(e.name)||SKIP.some(r=>r.test(e.name)))continue;
  const code=fs.readFileSync(f,'utf8');
  fs.writeFileSync(f,Obf.obfuscate(code,OPT).getObfuscatedCode());
}
walk(dir); console.log('✅ bundle obfusqué, source maps supprimées');
```

Annexe H — En-têtes de sécurité web

En-tête	Valeur
Strict-Transport-Security	max-age=31536000; includeSubDomains; preload
Content-Security-Policy	default-src 'self'; object-src 'none'; frame-ancestors 'none'; base-uri 'self'
X-Content-Type-Options	nosniff
X-Frame-Options	DENY
Referrer-Policy	same-origin
Permissions-Policy	camera=(), microphone=(), geolocation=(), payment=()
Cross-Origin-Opener-Policy	same-origin

Éviter 'unsafe-eval' / 'unsafe-inline' sur script-src (nonces). Tester en Report-Only d'abord. JWT en cookie HttpOnly; Secure; SameSite=Strict.

```
# EST versionné, mais SANS valeurs. Le vrai .env n'est JAMAIS versionné.  
# En prod : valeurs depuis gestionnaire de secrets / env CI, pas un fichier.  
# Tout NEXT_PUBLIC_* / VITE_* est PUBLIC : n'y mettez JAMAIS de secret.  
  
DJANGO_SECRET_KEY=  
DATABASE_URL=  
JWT_SIGNING_KEY=  
REDIS_URL=  
SMS_PROVIDER_TOKEN=  
PAYMENT_PROVIDER_SECRET=  
  
NEXT_PUBLIC_API_BASE_URL=  
NEXT_PUBLIC_APP_ENV=
```

Annexe J — Checklist Pré-Prod Go / No-Go

À remplir et **signer** avant chaque release. Une seule case « No » = No-Go.

App : _____ **Version :** _____ **Date :** _____ **Responsable :** _____

1 · NO SECRET

- ☐ gitleaks (code + historique) → 0 secret
- ☐ trufflehog → 0 secret vérifié
- ☐ scan binaire (apkileaks / dist) → 0 secret
- ☐ secrets en gestionnaire/env runtime, pas dans l'artefact ; `.env` non versionné
- ☐ aucune clé privée embarquée côté client

2 · NO DEBUG

- ☐ `check_no_debug.sh` → vert
- ☐ release signée clé de release (pas debug), clé hors repo
- ☐ `debuggable=false` / `allowBackup=false` / `DEBUG=False`
- ☐ pas de source map publique ; symboles Flutter archivés hors-app
- ☐ pas de log verbeux ni de PII

3 · SCAN

- ☐ gate CI `secure-gate` / `gate` → vert
- ☐ SAST → 0 ERROR ; SCA → 0 CRITICAL/HIGH non traité
- ☐ protection de branche active (PR + review + check requis)

4 · OBFUSCATION

- ☐ Flutter `--obfuscate --split-debug-info` + R8 ; Web minify + obfuscation bundles sensibles
- ☐ RASP / App Check actif (mobile) ; coût perf mesuré

5 · CHIFFREMENT & DURCISSEMENT

- ☐ TLS 1.3 + cert pinning ; algos imposés (aucun proscrié)
- ☐ Keystore/Keychain + SQLCipher ; clés serveur en KMS
- ☐ rate-limit + anti-rejeu + isolation tenant + en-têtes web + audit log

Verdict : ☐ **GO** (toutes cochées) ☐ **NO-GO**

Signature responsable : _____ **Signature revue sécurité :** _____

Qalami Secure Build Standard v1.0 — document vivant, à mettre à jour après chaque incident et chaque audit majeur. Distribué à l'équipe Qalami. Les versions d'outils citées (gitleaks, semgrep, actions) sont à vérifier/bumper au moment de l'adoption.